

Kubernetes Monitoring Stack

Installation and Alerting Configuration Guide

Longhorn | Prometheus | Grafana | Alert manager

Purpose

This guide explains the complete installation and configuration of the Kubernetes monitoring and alerting stack, including external access, persistent storage, metric collection, dashboards, alert rules, and email notifications.

- NGINX Ingress Controller
- Longhorn Storage
- Prometheus
- Grafana
- Alertmanager
- Node Exporter
- Kube State Metrics
- Grafana Alert Rules
- Email Notifications
- Dashboard Import

Environment Preparation Checklist

1. Grafana SMTP details
2. Grafana admin password
3. Grafana root URL
4. StorageClass name
5. PVC sizes
6. Grafana NodePort
7. Prometheus retention period

Step 1 - Verify Kubernetes Cluster

Verify all nodes are healthy and in Ready state

```
kubectl get nodes
```

Expected Output

Name	Status	Roles
master01	Ready	control-plane
worker01	Ready	worker
worker02	Ready	worker

Step 2 - Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
helm version
```

Verify Helm is installed successfully before proceeding.

Step 3 - Install Nginx Ingress Controller

Purpose

Provides external access to services running inside Kubernetes.

Installation Commands

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx \
  -n ingress-nginx \
  --create-namespace
```

Verify Installation

```
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Ensure all pods are running successfully.

Step 4 - Install Longhorn Storage

Purpose

Longhorn provides persistent storage for Grafana, Prometheus, and Alertmanager.

Installation Commands

```
helm repo add longhorn https://charts.longhorn.io
helm repo update
helm install longhorn longhorn/longhorn \
  -n longhorn-system \
  --create-namespace
```

Verify Longhorn Installation

```
kubectl get pods -n longhorn-system
kubectl get sc
```

Verify Longhorn StorageClass appears in output.

Example Output

Name	Provisioner
longhorn (default)	driver.longhorn.io

Step 5 - Add Prometheus Community Repository

```
helm repo add prometheus-community \
https://prometheus-community.github.io/helm-charts
helm repo update
```

Step 6 - Create Monitoring Namespace

```
kubectl create namespace monitoring
```

```
kubectl get ns monitoring
```

Step 7 - Create Values File

```
vi monitoring-values.yaml
```

Paste the following configuration into the file:

monitoring-values.yaml Configuration

```
grafana:
  service:
    type: NodePort
    nodePort: 32000
  adminUser: admin
  adminPassword: Admin@123
  persistence:
    enabled: true
    storageClassName: longhorn
    accessModes:
      - ReadWriteOnce
    size: 5Gi
  grafana.ini:
    server:
      root_url: http://<NODE_IP>:32000
    users:
      allow_sign_up: false
    auth.anonymous:
      enabled: false
    smtp:
      enabled: true
      host: smtp.office365.com:587
      user: monitoring@company.com
      password: ChangeMe
      from_address: monitoring@company.com
      from_name: Grafana
      skip_verify: true
    defaultDashboardsEnabled: false
  dashboardsProvider:
    enabled: false
  dashboardsConfigMaps: {}
  sidecar:
    dashboards:
      enabled: false
    datasources:
      enabled: false
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi
  alertmanager:
    alertmanagerSpec:
      storage:
        volumeClaimTemplate:
```

```

spec:
  storageClassName: longhorn
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
prometheus:
  prometheusSpec:
    retention: 7d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: longhorn
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 10Gi
  resources:
    requests:
      cpu: 500m
      memory: 2Gi
    limits:
      cpu: 2
      memory: 4Gi
prometheus-node-exporter:
  enabled: true
kube-state-metrics:
  enabled: true
prometheusOperator:
  resources:
    requests:
      cpu: 200m
      memory: 256Mi
    limits:
      cpu: 500m
      memory: 512Mi

```

Step 8 - Install Kube Prometheus Stack

```

helm install kube-prometheus-stack \
prometheus-community/kube-prometheus-stack \
-n monitoring \
-f monitoring-values.yaml

```

Wait until all pods become Running. This may take several minutes.

Step 9 - Verify Installation

Verification Commands

```

kubectl get pods -n monitoring
kubectl get svc -n monitoring
kubectl get pvc -n monitoring
kubectl get prometheus -n monitoring -o yaml | grep retention

```

Expected Components

- Grafana Pod - Running
- Prometheus Pod - Running
- Alertmanager Pod - Running
- Node Exporter Pods - Running
- Kube State Metrics - Running
- All PVCs - Bound

Step 10 - Access Grafana

```
kubect! get svc kube-prometheus-stack-grafana -n monitoring
```

Access URL Example

`http://10.10.10.10:32000`

Replace 10.10.10.10 with your Kubernetes node IP address.

Step 11 - Retrieve Grafana Admin Password

```
kubect! get secret kube-prometheus-stack-grafana -n monitoring \
-o jsonpath="{.data.admin-password}" \
| base64 -d ; echo
```

Default Credentials

- Username: admin
- Password: (Retrieved from secret as shown above)

Step 12 - Import Kubernetes Monitoring Dashboard

Dashboard Details

- Dashboard ID: 15661
- Dashboard Name: Kubernetes Monitoring Dashboard

Navigation Steps

8. Log in to Grafana
9. Navigate to Dashboards → Import
10. Enter Dashboard ID: 15661
11. Click Load
12. Select Prometheus as the Datasource
13. Click Import

The dashboard will now be available in the Dashboards section.

Alerting Configuration

This section covers the complete configuration of alerting in Grafana, including contact points, notification templates, and alert rules.

Step 13 - Verify SMTP Configuration

kubectl logs deployment/kube-prometheus-stack-grafana -n monitoring

Confirm that the SMTP configuration has loaded successfully in the logs.

Step 14 - Create Contact Point

Navigation

14. Grafana → Alerts → Contact Points
15. Click New Contact Point

Contact Point Configuration

- Name: Email-Team
- Integration: Email
- Recipients:
 - monitoring@company.com
 - admin@company.com
 - devops@company.com
- Optional Settings: Single Email - Enabled

Verification

16. Click Test to verify email delivery
17. Verify that test email is received

Save Contact Point

Click Save to save the contact point.

Step 15 - Create Notification Template

Navigation

18. Grafana → Alerts → Notification Templates
19. Click New Template

Template Configuration

Template Name: ClientName_Tech_Email_Template

Template Content:

```
{{ define "ClientName_Tech_Email_Template" -}}
Alert Status : {{ .Status | toUpper }}
Environment : {{ or (index .CommonLabels "environment") "N/A" }}
Alert Type   : {{ or (index .CommonLabels "alerttype") "N/A" }}
Alert Name   : {{ or (index .CommonLabels "alertname") "N/A" }}
Severity     : {{ or (index .CommonLabels "severity") "N/A" }}
-----
Namespace    : {{ or (index .CommonLabels "namespace") "N/A" }}
Pod Name     : {{ or (index .CommonLabels "pod") "N/A" }}
{{- if index .CommonLabels "reason" }}
Failure Reason : {{ index .CommonLabels "reason" }}
{{- end }}
{{- if index .CommonLabels "container" }}
Container      : {{ index .CommonLabels "container" }}
{{- end }}
-----
{{- if index .CommonLabels "node" }}
Node Name      : {{ index .CommonLabels "node" }}
```

```

{{- end }}
{{- if index .CommonLabels "host_ip" }}
Node IP      : {{ index .CommonLabels "host_ip" }}
{{- end }}
{{- if index .CommonLabels "pod_ip" }}
Pod IP       : {{ index .CommonLabels "pod_ip" }}
{{- end }}
-----
{{- if index .CommonLabels "service" }}
Service      : {{ index .CommonLabels "service" }}
{{- end }}
{{- if index .CommonLabels "job" }}
Job          : {{ index .CommonLabels "job" }}
{{- end }}
Instance/IP   : {{ or (index .CommonLabels "instance") "N/A" }}
-----
Triggered At
{{- range .Alerts }}
{{ .StartsAt }}
{{- end }}
{{- end }}

```

Save Template

Click Save to save the notification template.

Step 16 - Configure Contact Point Template

Navigation

20. Grafana → Alerts → Contact Points
21. Click on Email-Team contact point

Email Configuration

Subject Line:

```

{{ .Status | toUpper }} | Grafana | Client-ID | {{ .CommonLabels.environment }} | {{
.CommonLabels.alerttype }} | {{ .CommonLabels.alertname }}

```

Example Subject

FIRING | Grafana | Client-ID | Production | Infrastructure | CPU Usage High

Message Template

```

{{ template "ClientName_Tech_Email_Template" . }}

```

Save Contact Point

Click Save to save the contact point configuration.

Alert Rules Configuration

This section defines all the alert rules to be created in Grafana. Each alert is configured with specific query conditions, labels, and annotations.

Alert 1 - GPU Not Reporting / Exporter Down

Alert Name	GPU Not Reporting Exporter Down
-------------------	-----------------------------------

Query	up{job="scrapeConfig/monitoring/external-ai-gpu-exporter"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Infrastructure
Labels	environment = Production alerttype = GPU severity = Critical team = DevOps
Summary	GPU Exporter Down
Description	GPU exporter is not reporting metrics from {{ \$labels.instance }}.
Contact Point	Email-Team

Alert 2 - CPU Usage High

Alert Name	CPU Usage High
Query	100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Condition	Is Above 80
Duration	5 Minutes
Folder	Infrastructure
Labels	environment = Production alerttype = Infrastructure severity = Critical team = DevOps
Summary	CPU Usage High
Description	CPU usage on {{ \$labels.instance }} exceeded 80%.
Contact Point	Email-Team

Alert 3 - Memory Usage High

Alert Name	Memory Usage High
Query	node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100
Condition	Is Below 20
Duration	5 Minutes
Folder	Infrastructure
Labels	environment = Production alerttype = Infrastructure severity = Critical team = DevOps
Summary	Memory Usage High
Description	Memory utilization on {{ \$labels.instance }} exceeded 80%.

Contact Point	Email-Team
----------------------	------------

Alert 4 - Disk Usage High

Alert Name	Disk Usage High
Query	node_filesystem_avail_bytes / node_filesystem_size_bytes * 100
Condition	Is Below 20
Duration	5 Minutes
Folder	Infrastructure
Labels	environment = Production alerttype = Infrastructure severity = Critical team = DevOps
Summary	Disk Usage High
Description	Disk utilization on {{ \$labels.instance }} exceeded 80%.
Contact Point	Email-Team

Alert 5 - Node Not Ready

Alert Name	Node Not Ready
Query	kube_node_status_condition{condition="Ready",status="true"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Kubernetes
Labels	environment = Production alerttype = Kubernetes severity = Critical team = DevOps
Summary	Node Not Ready
Description	Node {{ \$labels.node }} is not reporting Ready status.
Contact Point	Email-Team

Alert 6 - Node Scheduling Disabled

Alert Name	Node Scheduling Disabled
Query	kube_node_spec_unschedulable
Condition	Is Above 0
Duration	5 Minutes
Folder	Kubernetes

Labels	environment = Production alerttype = Kubernetes severity = Warning team = DevOps
Summary	Node Scheduling Disabled
Description	Node {{ \$labels.node }} has been marked unschedulable.
Contact Point	Email-Team

Alert 7 - Pod Restart High

Alert Name	Pod Restart High
Query	rate(kube_pod_container_status_restarts_total[15m]) > 0.1
Condition	Is Above 0
Duration	5 Minutes
Folder	Kubernetes
Labels	environment = Production alerttype = Kubernetes severity = Warning team = DevOps
Summary	Pod Restart High
Description	Container restart count exceeded configured threshold.
Contact Point	Email-Team

Alert 8 - Pod Not Running

Alert Name	Pod Not Running
Query	kube_pod_status_phase{phase="Running"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Kubernetes
Labels	environment = Production alerttype = Kubernetes severity = Critical team = DevOps
Summary	Pod Not Running
Description	Pod {{ \$labels.pod }} in namespace {{ \$labels.namespace }} is not running.
Contact Point	Email-Team

Alert 9 - PVC Usage High

Alert Name	PVC Usage High
-------------------	----------------

Query	kubelet_volume_stats_available_bytes / kubelet_volume_stats_capacity_bytes * 100
Condition	Is Below 20
Duration	5 Minutes
Folder	Storage
Labels	environment = Production alerttype = Storage severity = Critical team = DevOps
Summary	PVC Usage High
Description	PVC utilization exceeded configured threshold.
Contact Point	Email-Team

Alert 10 - PV Not Bound

Alert Name	PV Not Bound
Query	kube_persistentvolume_status_phase{phase="Bound"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Storage
Labels	environment = Production alerttype = Storage severity = Critical team = DevOps
Summary	PV Not Bound
Description	Persistent Volume is not in Bound state.
Contact Point	Email-Team

Alert 11 - Jenkins Server Down

Alert Name	Jenkins Server Down
Query	up{job="jenkins"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Application
Labels	environment = Production alerttype = Application severity = Critical team = DevOps
Summary	Jenkins Server Down
Description	Jenkins node exporter is unreachable.

Contact Point	Email-Team
----------------------	------------

Alert 12 - Harbor Server Down

Alert Name	Harbor Server Down
Query	up{job="harbor"}
Condition	Is Below 1
Duration	5 Minutes
Folder	Application
Labels	environment = Production alerttype = Application severity = Critical team = DevOps
Summary	Harbor Server Down
Description	Harbor node exporter is unreachable.
Contact Point	Email-Team

Verify Alert Status

Check Alert Rules

22. Navigate to Grafana → Alerts → Alert Rules
23. Verify all alerts appear in the list
24. Check status of each alert

Expected Alert Status

- Normal - Alert condition is not met
- Pending - Alert condition met but threshold time not reached
- Firing - Alert condition met and threshold time exceeded

Troubleshooting

Useful Kubernetes Commands

Check pod status:

```
kubectl get pods -n monitoring
```

Check services:

```
kubectl get svc -n monitoring
```

Check PVC status:

```
kubectl get pvc -n monitoring
```

Check ingress resources:

```
kubectl get ingress -n monitoring
```

Check storage classes:

```
kubectl get sc
```

Describe a specific PVC:

```
kubectl describe pvc <PVC_NAME> -n monitoring
```

View Grafana logs:

```
kubectl logs deployment/kube-prometheus-stack-grafana -n monitoring
View Prometheus logs:
kubectl logs prometheus-kube-prometheus-stack-prometheus-0 -n monitoring
```

Prometheus Target Validation

Follow these steps to validate that all Prometheus targets are correctly configured:

Step 1: Port Forward to Prometheus

```
kubectl port-forward svc/kube-prometheus-stack-prometheus 9090:9090 -n monitoring
```

Step 2: Access Prometheus Web UI

Open your browser and navigate to:

<http://localhost:9090/targets>

Step 3: Verify Target Status

- All targets should show UP status (green)
- Check labels for each target
- Verify scrape intervals are correct
- Review last scrape time for each target

Longhorn Validation

Verify that Longhorn storage is functioning correctly:

Check Longhorn Pods

```
kubectl get pods -n longhorn-system
```

All Longhorn pods should be Running.

Check PVC Status

```
kubectl get pvc -n monitoring
```

All PVCs should show Bound status.

Detailed PVC Information

```
kubectl describe pvc <PVC_NAME> -n monitoring
```

This command provides detailed information about a specific PVC, including:

- Capacity
- Access modes
- Persistent Volume association
- Volume claims information
- Events and status